

Complete ML Algorithms Guide with Code

SUPERVISED LEARNING ALGORITHMS

1. LINEAR REGRESSION (Ridge & LASSO)

What it does: Finds the best straight line, Ridge/LASSO prevent overfitting **When to use:** Predicting continuous numbers with regularization

```
python

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
import numpy as np

# Sample data
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Regular Linear Regression
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Ridge Regression (L2 regularization)
ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X_train, y_train)

# LASSO Regression (L1 regularization)
lasso_model = Lasso(alpha=1.0)
lasso_model.fit(X_train, y_train)

# Make predictions
predictions = ridge_model.predict(X_test)
print(f"Ridge Predictions: {predictions}")
```

Memory Tip: Ridge = smooth curve, LASSO = feature selection (makes some features zero)

2. LOGISTIC REGRESSION

What it does: Classifies things into categories using probability **When to use:** Binary or multi-class classification

python

```
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
```

Create sample data

```
X, y = make_classification(n_samples=100, n_features=2, n_classes=2)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Create and train model

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

Make predictions

```
predictions = model.predict(X_test)
probabilities = model.predict_proba(X_test)
```

```
print(f"Accuracy: {model.score(X_test, y_test)}")
```

```
print(f"Probabilities: {probabilities}")
```

Memory Tip: Logistic = Logic/Categories, S-shaped curve between 0 and 1

3. DECISION TREE

What it does: Creates a tree of yes/no questions **When to use:** When you need interpretable results

python

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
```

Load data

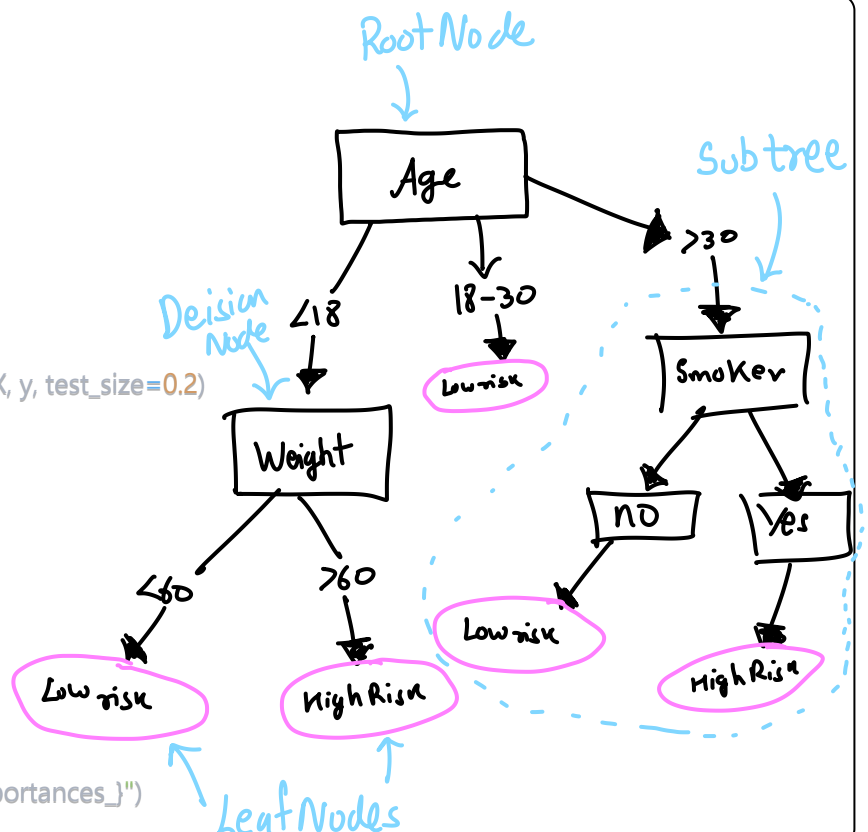
```
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Create and train model

```
model = DecisionTreeClassifier(max_depth=3)
model.fit(X_train, y_train)
```

Make predictions

```
predictions = model.predict(X_test)
print(f"Accuracy: {model.score(X_test, y_test)}")
print(f"Feature importance: {model.feature_importances_}")
```



Memory Tip: Like a flowchart - asks questions and follows branches

4. ADABOOST

What it does: Combines weak learners, focuses on mistakes from previous models **When to use:** When you want to boost performance of weak classifiers

```
python

from sklearn.ensemble import AdaBoostClassifier

# Create and train model
model = AdaBoostClassifier(n_estimators=50, learning_rate=1.0)
model.fit(X_train, y_train)

# Make predictions
predictions = model.predict(X_test)
print(f"Accuracy: {model.score(X_test, y_test)}")
```

Memory Tip: Ada = Adaptive, learns from mistakes and focuses on hard examples

5. RANDOM FOREST

What it does: Creates many decision trees and averages their votes **When to use:** When you want high accuracy and robustness

```
python

from sklearn.ensemble import RandomForestClassifier

# Create and train model
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Make predictions
predictions = model.predict(X_test)
print(f"Accuracy: {model.score(X_test, y_test)}")
print(f"Feature importance: {model.feature_importances_}")
```

Memory Tip: Many trees = forest, each tree votes for final decision

6. GRADIENT BOOSTING

What it does: Builds models sequentially, each correcting previous errors **When to use:** High accuracy competitions, complex patterns

```
python
```

```
from sklearn.ensemble import GradientBoostingClassifier

# Create and train model
model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1)
model.fit(X_train, y_train)

# Make predictions
predictions = model.predict(X_test)
print(f"Accuracy: {model.score(X_test, y_test)}")
```

Memory Tip: Gradient = direction of improvement, each model fixes previous mistakes

7. XGBOOST

What it does: Extreme Gradient Boosting - faster and better than regular boosting **When to use:** Competitions, structured data, high performance needed

```
python

import xgboost as xgb

# Create and train model
model = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1)
model.fit(X_train, y_train)

# Make predictions
predictions = model.predict(X_test)
print(f"Accuracy: {model.score(X_test, y_test)}")

# Feature importance
import matplotlib.pyplot as plt
xgb.plot_importance(model)
plt.show()
```

Memory Tip: XG = eXtreme Gradient, the supercharged version of gradient boosting

8. NAIVE BAYES

What it does: Uses probability and assumes features are independent **When to use:** Text classification, spam detection, when features are independent

```
python
```

```
from sklearn.naive_bayes import GaussianNB

# Create and train model
model = GaussianNB()
model.fit(X_train, y_train)

# Make predictions
predictions = model.predict(X_test)
probabilities = model.predict_proba(X_test)

print(f"Accuracy: {model.score(X_test, y_test)}")
```

Memory Tip: Naive = assumes independence, Bayes = uses probability rules

9. SVM (Support Vector Machine)

What it does: Finds the best boundary to separate classes **When to use:** Text classification, when you have clear margins

```
python

from sklearn.svm import SVC

# Create and train model
model = SVC(kernel='rbf', probability=True)
model.fit(X_train, y_train)

# Make predictions
predictions = model.predict(X_test)
print(f"Accuracy: {model.score(X_test, y_test)}")
```

Memory Tip: SVM = finds the best fence with maximum margin between groups

10. K-NEAREST NEIGHBORS (KNN)

What it does: Looks at k closest neighbors to make decisions **When to use:** Simple classification, recommendation systems

```
python
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
# Create and train model
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
model.fit(X_train, y_train)
```

```
# Make predictions
```

```
predictions = model.predict(X_test)
```

```
print(f"Accuracy: {model.score(X_test, y_test)}")
```

Memory Tip: "Tell me who your neighbors are, I'll tell you who you are"

UNSUPERVISED LEARNING ALGORITHMS

1. K-MEANS CLUSTERING

What it does: Groups data into k clusters **When to use:** Customer segmentation, finding hidden groups

```
python
```

```
from sklearn.cluster import KMeans
```

```
import matplotlib.pyplot as plt
```

```
# Create sample data
```

```
X, _ = make_classification(n_samples=100, n_features=2, n_centers=3)
```

```
# Create and train model
```

```
model = KMeans(n_clusters=3)
```

```
clusters = model.fit_predict(X)
```

```
# Plot results
```

```
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis')
```

```
plt.scatter(model.cluster_centers[:, 0], model.cluster_centers[:, 1],  
            c='red', marker='x', s=200)
```

```
plt.title('K-Means Clustering')
```

```
plt.show()
```

```
print(f"Cluster centers: {model.cluster_centers_}")
```

Memory Tip: K groups, finds the mean center of each group

2. DBSCAN

What it does: Finds clusters of any shape, identifies outliers **When to use:** When clusters have irregular shapes, when you have noise

python

```
from sklearn.cluster import DBSCAN

# Create and train model
model = DBSCAN(eps=0.5, min_samples=5)
clusters = model.fit_predict(X)

# Plot results
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis')
plt.title('DBSCAN Clustering')
plt.show()

print(f"Number of clusters: {len(set(clusters)) - (1 if -1 in clusters else 0)}")
print(f"Number of outliers: {list(clusters).count(-1)}")
```

Memory Tip: DB = Density-Based, finds dense areas and marks sparse points as outliers

3. HIERARCHICAL CLUSTERING

What it does: Creates a tree of clusters (dendrogram) **When to use:** When you want to see cluster relationships at different levels

python

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Create and train model
model = AgglomerativeClustering(n_clusters=3)
clusters = model.fit_predict(X)

# Create dendrogram
linkage_matrix = linkage(X, method='ward')
plt.figure(figsize=(10, 6))
dendrogram(linkage_matrix)
plt.title('Hierarchical Clustering Dendrogram')
plt.show()

# Plot clusters
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis')
plt.title('Hierarchical Clustering')
plt.show()
```

Memory Tip: Like a family tree of clusters, shows relationships at different levels

4. PCA (Principal Component Analysis)

What it does: Reduces dimensions while keeping important information **When to use:** Visualization, reducing features, removing correlation

```
python

from sklearn.decomposition import PCA

# Create sample high-dimensional data
X_high_dim = np.random.rand(100, 10) # 10 features

# Create and train model
model = PCA(n_components=2) # Reduce to 2 dimensions
X_reduced = model.fit_transform(X_high_dim)

# Plot results
plt.scatter(X_reduced[:, 0], X_reduced[:, 1])
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.title('PCA: 10D data reduced to 2D')
plt.show()

print(f"Explained variance ratio: {model.explained_variance_ratio_}")
print(f"Total variance explained: {sum(model.explained_variance_ratio_):.2f}")
```

Memory Tip: PCA = finds the most important directions in your data

5. LDA (Linear Discriminant Analysis)

What it does: Reduces dimensions while maximizing class separation **When to use:** Dimensionality reduction for classification, when you have labels

```
python
```



```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
# Need labeled data for LDA
```

```
iris = load_iris()
```

```
X, y = iris.data, iris.target
```

```
# Create and train model
```

```
model = LinearDiscriminantAnalysis(n_components=2)
```

```
X_lda = model.fit_transform(X, y)
```

```
# Plot results
```

```
plt.scatter(X_lda[:, 0], X_lda[:, 1], c=y, cmap='viridis')
```

```
plt.xlabel('First Linear Discriminant')
```

```
plt.ylabel('Second Linear Discriminant')
```

```
plt.title('LDA: Dimensionality Reduction')
```

```
plt.show()
```

```
print(f"Explained variance ratio: {model.explained_variance_ratio_}")
```

Memory Tip: LDA = like PCA but considers class labels to maximize separation

QUICK ALGORITHM SELECTION GUIDE

For Classification:

- **Start with:** Logistic Regression
- **High accuracy:** Random Forest, XGBoost
- **Interpretable:** Decision Tree, Naive Bayes
- **Text data:** Naive Bayes, SVM
- **Small dataset:** KNN
- **Need probability:** Logistic Regression, Naive Bayes

For Regression:

- **Start with:** Linear Regression
- **Prevent overfitting:** Ridge, LASSO
- **High accuracy:** Random Forest, XGBoost, Gradient Boosting
- **Interpretable:** Decision Tree

For Clustering:

- **Round clusters:** K-Means
- **Any shape clusters:** DBSCAN

- **Hierarchical structure:** Hierarchical Clustering

For Dimensionality Reduction:

- **Visualization:** PCA
- **Classification prep:** LDA
- **Keep important features:** PCA

Basic Workflow Template:

```
python

# 1. Import and prepare data
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 2. Scale data (for SVM, KNN, Neural Networks)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. Create and train model
model = YourChosenAlgorithm()
model.fit(X_train_scaled, y_train)

# 4. Make predictions and evaluate
predictions = model.predict(X_test_scaled)
from sklearn.metrics import accuracy_score, classification_report
print(f"Accuracy: {accuracy_score(y_test, predictions)}")
print(classification_report(y_test, predictions))
```

Installation commands:

```
bash

pip install scikit-learn xgboost matplotlib seaborn pandas numpy
```

Remember: Start simple, then increase complexity if needed!